

Snake Stack

CS108 — Software Systems Lab

Project Report

Sajal Sahu Himaanshu

April 29, 2026

Contents

1	External Libraries	2
2	Design & Architecture	2
2.1	Three-Layer Architecture	2
2.2	Data Format	2
2.3	Game Engine Design	3
3	Features Implemented	3
3.1	Frontend Features	3
3.2	Backend Features	4
3.3	Bash Admin Features	4
4	Directory Structure	5
5	Installation & Setup	6
5.1	Prerequisites	6
5.2	Setup Steps	6
6	Usage	6
6.1	Starting the Server	6
6.2	Playing the Game	7
6.3	Using the Admin Script	7
7	Retrospective	8
7.1	Challenges Faced	8
7.2	What We Would Do Differently With More Time	10
8	References	10

1 External Libraries

The following external libraries and tools were used in the project:

Library/Tool	Version	Purpose
Flask	3.1.1	Python web framework used to serve the game frontend and handle score
Bootstrap	5.3.0	CSS framework bundled locally for responsive layout and styling of modal
Python	3.10.12	Runtime for the Flask backend server.
Bash	5.x	Scripting language for the administration utility that processes the flat-file
LibreSprite	—	Pixel art editor used to create all game sprites (snake, food, borders, hear
bc	—	Bash calculator used for arithmetic operations in analytics (averages, fract
zip	—	Compression utility used for log rotation backup.

Table 1: External libraries and tools used in the project.

Standard Python libraries used: `datetime` (timestamp generation), `json` (implicit via Flask), `os` (file operations).

2 Design & Architecture

2.1 Three-Layer Architecture

The project follows a strict three-layer separation of concerns:

```
Browser (JS) --> POST /save_score --> Flask (Python) --> history.txt <-- Bash
```

1. **Layer 1 — Frontend (JavaScript + HTML5 Canvas):** Handles the entire game experience including rendering, input handling, collision detection, and score reporting. The frontend only knows the API endpoint URL (`/save_score`) and contains no file I/O or storage logic.
2. **Layer 2 — Backend (Python/Flask):** Serves the static game files and provides a REST endpoint for score submission. Flask validates incoming JSON payloads, generates server-side timestamps, and appends formatted entries to `history.txt`.
3. **Layer 3 — Administration (Bash):** Reads `history.txt` directly using text-processing tools (`awk`, `cut`, `sort`, `sed`, `grep`, `head`, `tail`). It operates independently of the Flask server.

2.2 Data Format

Each game result is stored as a single line in `history.txt`:

```
[2025-06-12 14:35:02] Herobrine | 42 | WALL | 85
```

Fields are separated by `' | '` (space-pipe-space). This delimiter was chosen because:

- Pipes are not common in usernames, preventing parsing ambiguity.
- The format is human-readable and easy to parse with `cut -d ' | '`.

- Square brackets around the timestamp make it visually distinct and parseable with regex.

2.3 Game Engine Design

The game engine uses a `Point` class to represent positions on the grid. The snake is stored as an array of `Point` objects, where each element has `x`, `y` coordinates and a `type` variable for sprite selection:

- Type 0 = Head
- Type 1 = Straight body
- Type 2 = Curved body
- Type 3 = Tail

Snake rendering at curves is determined by checking the previous and next point's positions relative to the current point. If neighbors are on the same axis, a straight sprite is used; otherwise, a curve sprite is rendered with appropriate rotation.

3 Features Implemented

3.1 Frontend Features

- **Start Modal:** Displays game rules, food type descriptions, control instructions, difficulty slider, and username input with validation (Figure 2).
- **Game Canvas:** 15×20 grid-based Snake game rendered on HTML5 Canvas with pixel art sprites created in Libresprite.
- **Controls:** Arrow keys and WASD support with direction change protection (prevents reversing into self within same tick).
- **Four Food Types:**
 - Green Apple — Health +1 (snake grows by 1)
 - Golden Apple — Health +3 (snake grows by 3)
 - Cookie — Barrier immunity for 10 seconds
 - Lightning — Time slow for 10 seconds (halves game speed)
- **Health System:** 3 lives for wall collisions, displayed as heart icons. Self-collision causes instant death.
- **Difficulty Slider:** Four levels — Easy (200ms), Medium (150ms), Hard (100ms), Insane (50ms).
- **Game Over Modal:** Shows death cause, random death message, score, time survived, session best score, timestamp, and a Play Again button (Figure 5).
- **Score Reporting:** Automatic POST request to `/save_score` on death via fetch API.

- **Session Best Score:** Tracked in a JavaScript variable across games within the same browser session.
- **Score:** Score is defined as the number of points gained from collecting powerups. It is always 4 greater than the snake's length, since length grows in step with score but the initial snake length is set to 4.
- **Point Class:** A `Point` class is defined to store `x`, `y`, and `type` fields, serving as the building block for both snake parts and powerup positions on the grid.
- **Snake Array:** All snake parts are stored in a `snake[]` array, where index 0 is always the head and subsequent indices represent body and tail segments in order.
- **Powerup Spawning with Probability:** A random number generator is used to determine which powerup spawns at a given time. Each powerup type is assigned a probability range — rarer powerups (such as lightning and cookie) have lower probabilities than common ones (such as the green apple), so they appear less frequently.
- **Type Field Usage:** The `type` field of a `Point` serves a dual purpose: for snake parts it encodes whether the segment is a head (0), straight body (1), curved body (2), or tail (3); for powerups it encodes which food type the point represents (1 = green apple, 2 = golden apple, 3 = cookie, 4 = lightning).
- **Type Determination for Snake Parts:** The `type` of each non-head snake part is decided by comparing the `x` and `y` coordinates of its preceding and succeeding points. If the preceding and succeeding points share the same `x` or the same `y`, the part is classified as straight (type 1); otherwise it is classified as curved (type 2). For curved parts, the position deltas between neighbours are further compared to determine the correct rotation angle for the curve sprite.
- **Collision Detection:** All three collision types are resolved by comparing the snake head's position against the relevant target. Powerup collision checks whether the head's coordinates match any powerup in the `powerups[]` array; self-collision checks whether the head's coordinates match any other element in `snake[]`; wall collision checks whether the head's `x` or `y` is at the grid boundary while moving toward it.

3.2 Backend Features

- **Static File Serving:** Flask serves `index.html` and all assets from the `static/` directory.
- **Score Endpoint:** POST `/save_score` validates JSON fields (`name`, `score`, `cause`, `duration`), generates a server-side timestamp, and appends to `history.txt`.
- **Input Validation:** Rejects requests with missing fields, empty names, or invalid cause values (must be `WALL` or `SELF`).

3.3 Bash Admin Features

- **User Selection:** Filter all queries to a specific user via a sub-menu.

- **Analytics:** Top 5 users, average score, average time survived, wall/self death fractions — computed using `awk`, `cut`, `bc`.
- **Recent Scores:** Paginated view (5 per page) with colored output.
- **Deletion:** By username, exact timestamp, before/after timestamp, or invalid records — using `sed -i` for deletion with confirmation prompts.
- **Log Rotation:** Backs up `history.txt` as a compressed zip and keeps only the last 10 entries.
- **Sorting:** By score, timestamp, or username.
- **ANSI Colors:** Wall deaths displayed in blue, self deaths in red, menu items in yellow/green/cyan.
- **Edge Case Handling:** Checks for missing or empty `history.txt` before starting.

4 Directory Structure

```
snake-system/  
  app.py # Flask backend server  
  admin.sh # Bash administration script  
  history.txt # Game score storage (auto-created)  
  requirements.txt # Python dependencies (flask==3.1.1)  
  README.md # Setup and usage documentation  
  report/  
    report.tex # LaTeX report source  
    Makefile # Compiles the report  
  static/  
    index.html # Main game page with modals  
    style.css # All CSS styles  
    engine.js # Game logic and rendering  
    bootstrap.min.css # Bootstrap CSS (bundled offline)  
    bootstrap.bundle.min.js # Bootstrap JS (bundled offline)  
    Apple.png # Green apple sprite  
    goldenApple.png # Golden apple sprite  
    cookie.png # Cookie sprite  
    lightning.png # Lightning powerup sprite  
    snakehead.png # Snake head sprite  
    snakebod.png # Snake body sprite  
    snakeCurve.png # Snake curve sprite  
    snaketail.png # Snake tail sprite  
    heart.png # Health indicator sprite  
    backtile2.png # Game background tile  
    borders.png # Game border overlay
```

5 Installation & Setup

5.1 Prerequisites

- Python 3.10.12
- pip
- A modern web browser (Chrome, Firefox, Edge)
- Bash shell (for admin script)
- bc, zip (for admin analytics and log rotation)

5.2 Setup Steps

```
# Clone the repository
git clone https://github.com/himaanshu0102/CS108-Project.git
cd CS108-Project/snake-system

# Create and activate virtual environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Start the Flask server
python3 app.py
```

The server starts on <http://127.0.0.1:5000>. Open this URL in a browser to play the game.

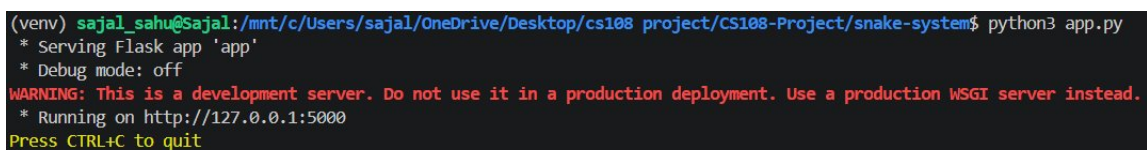
To run the admin script (does not require the server to be running):

```
bash admin.sh
```

6 Usage

6.1 Starting the Server

Run `python3 app.py` from the `snake-system/` directory. The terminal shows the Flask development server running on localhost (Figure 1).



```
(venv) sajal_sahu@Sajal:/mnt/c/Users/sajal/OneDrive/Desktop/cs108 project/CS108-Project/snake-system$ python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Figure 1: Flask server running on <http://127.0.0.1:5000>.

6.2 Playing the Game

1. Open `http://127.0.0.1:5000` in a browser.
2. The start modal appears with game rules, food types, controls, and a difficulty slider (Figure 2).
3. Enter your first name and select a difficulty level.
4. Click **Start Game** to begin (Figure 3).
5. Control the snake using WASD or Arrow Keys. Collect food to grow and increase your score (Figure 4).
6. When the snake dies, the game over screen displays your score, time, best score, cause of death, and a random death message (Figure 5).
7. Click **Play Again** to restart.

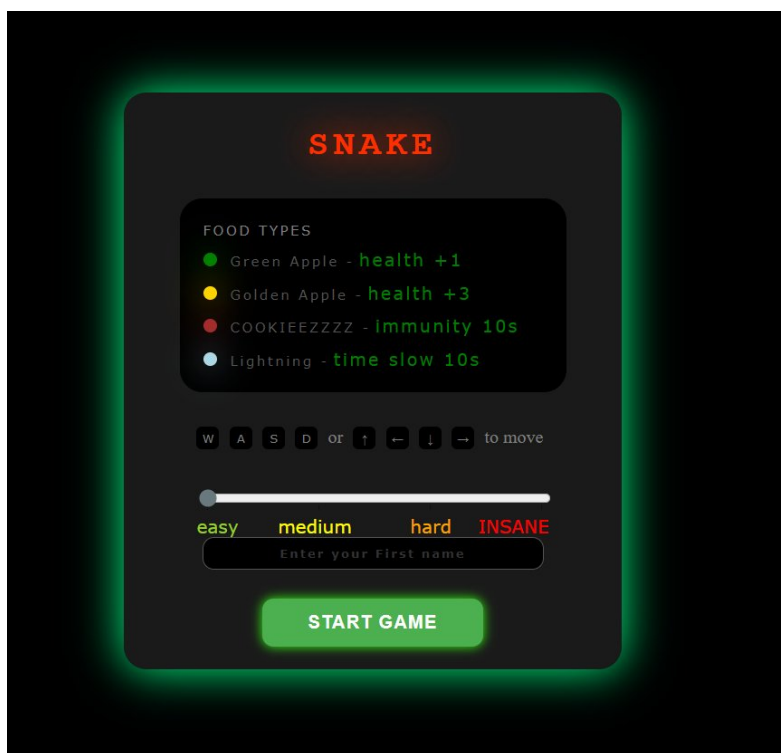


Figure 2: Start modal showing game rules, food types, controls, and difficulty slider.

6.3 Using the Admin Script

Run `bash admin.sh` from the `snake-system/` directory. The menu-driven interface appears with colored output (Figure 6).

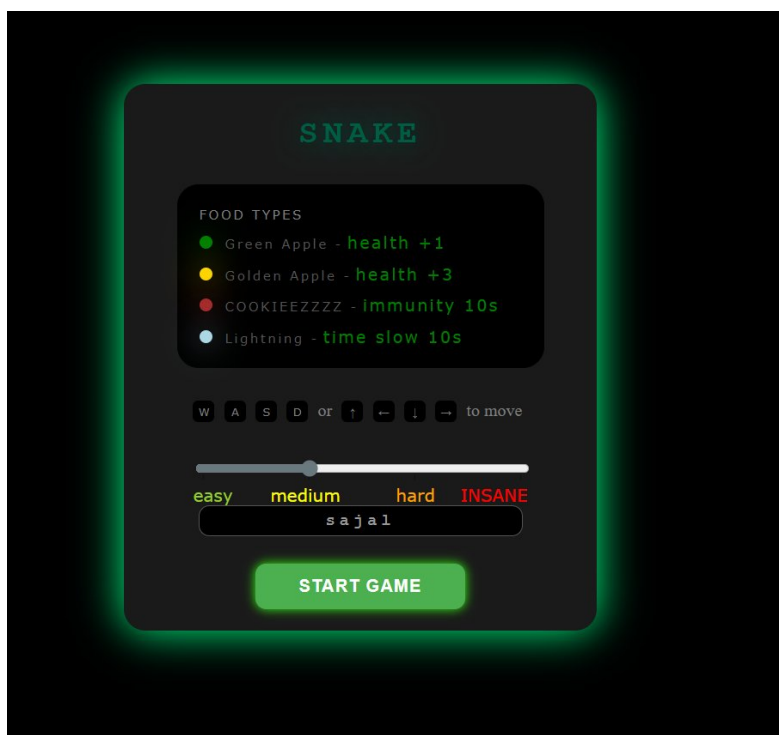


Figure 3: Player entering their name before starting the game.

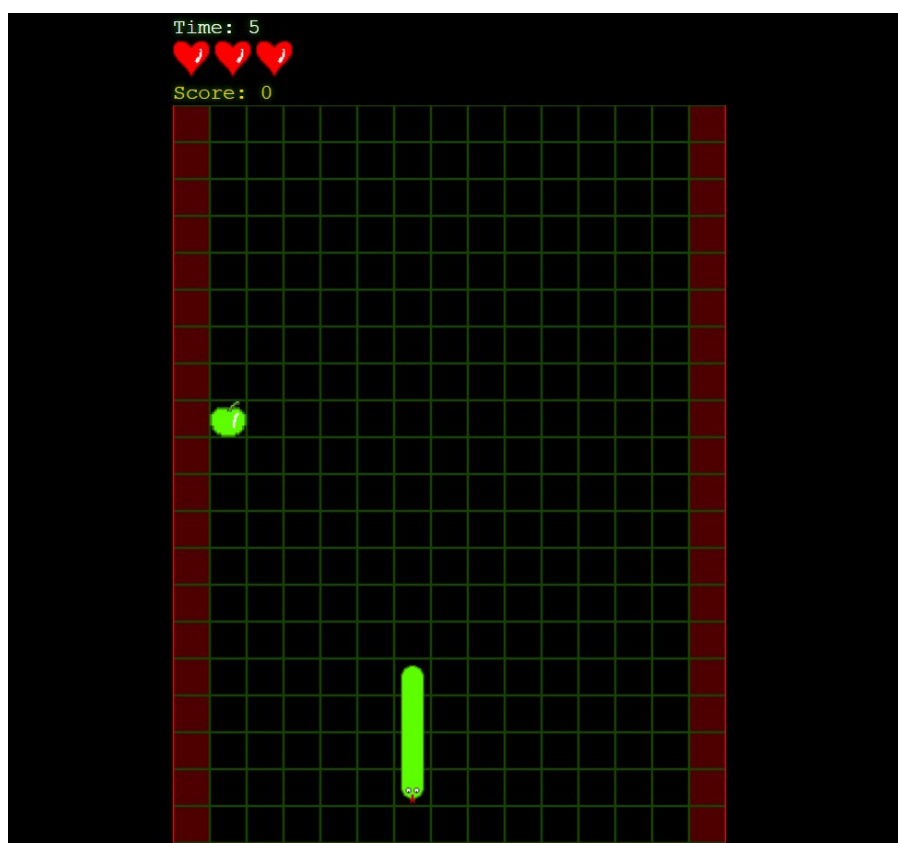


Figure 4: Gameplay showing the snake, food, health hearts, score, and timer.

7 Retrospective

7.1 Challenges Faced

1. **Snake curve rendering:** Determining the correct rotation for curve sprites when the snake turns was challenging. We solved it by computing the position delta

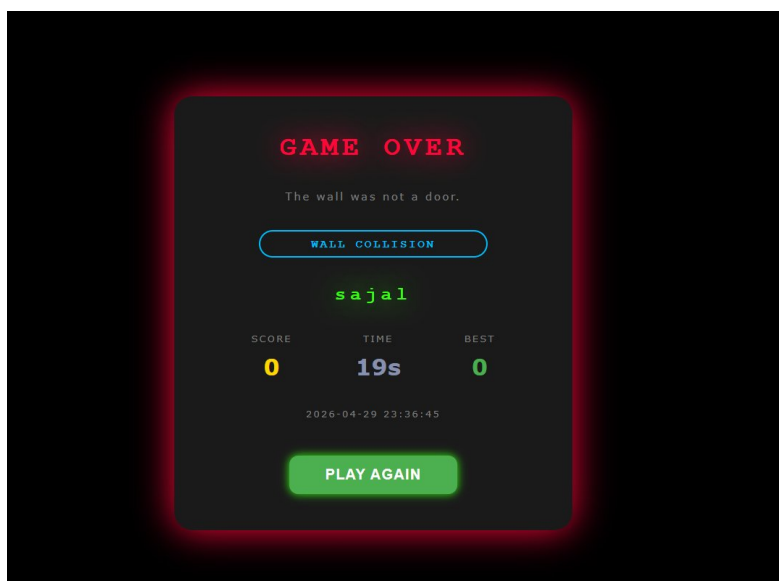


Figure 5: Game over screen with death cause, score, time, best score, and timestamp.

```
(venv) sajal_sahu@Sajal:/mnt/c/Users/sajal/OneDrive/Desktop/cs108 project/CS108-Project/snake-system$ bash admin.sh
```

Figure 6: Running the bash admin script.

```
(venv) sajal_sahu@Sajal:/mnt/c/Users/sajal/OneDrive/Desktop/cs108 project/CS108-Project/snake-system$ bash admin.sh
=====
Snake Admin Menu
=====
1. Select User Name
2. View Analytics
3. View Recent Scores
4. Delete User Data
5. Log Rotation
6. Sort data
7. Exit
Enter your choice:
|
```

Figure 7: Bash admin script menu with ANSI colored output.

between the previous and next points relative to the current point, and mapping each combination to a rotation angle.

2. **Wall wrap-around vs. wall death:** The initial implementation allowed the snake to wrap around walls. We redesigned this to implement a health-based wall collision system where the snake loses one of three hearts on each wall hit, dying only when all hearts are depleted.
3. **Bash deletion with special characters:** The `history.txt` format contains square brackets and pipes, which are special characters in `sed` regex. We solved this by escaping all special characters before passing lines to `sed` for deletion.
4. **Preserving other users' data during user-specific deletion:** Initially, deletion operations overwrote `history.txt` with only the filtered user's data. We fixed this by using `sed -i` to delete only the specific lines, keeping other users' records intact.
5. **Multiple direction changes per game tick:** Without a `nextDirection` buffer,

rapid key presses could cause the snake to reverse into itself. We implemented a direction queue that only applies the latest input at the start of each game tick.

6. **Offline Bootstrap:** The project must run without internet access. We bundled Bootstrap's minified CSS and JS files locally in the `static/` folder instead of using CDN links.

7.2 What We Would Do Differently With More Time

1. **Leaderboard page:** Add a separate route that displays a formatted leaderboard fetched from `history.txt`, visible in the browser.
2. **Sound effects:** Add audio feedback for eating food, dying, and collecting powerups using the Web Audio API.
3. **Smoother animations:** Implement interpolation-based movement instead of grid-snapping for a smoother visual experience.
4. **Mobile support:** Add touch/swipe controls so the game is playable on phones and tablets.
5. **More powerups:** Add additional food types such as speed boost, score multiplier, or obstacles that spawn over time.
6. **Persistent high scores:** Fetch the user's all-time best score from the server on login, rather than tracking only the current session.

8 References

1. Flask Documentation — <https://flask.palletsprojects.com/en/stable/> [13]
2. Flask Quickstart Guide — <https://flask.palletsprojects.com/en/stable/quickstart/> [14]
3. MDN Web Docs: Canvas API — https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API [8]
4. MDN Web Docs: Fetch API — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API [9]
5. MDN Web Docs: KeyboardEvent — <https://developer.mozilla.org/en-US/docs/Web/API/KeyboardEvent> [11]
6. MDN Web Docs: setInterval — <https://developer.mozilla.org/en-US/docs/Web/API/setInterval> [12]
7. MDN Web Docs: JSON.stringify — https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify [10]
8. Bootstrap 5 Documentation — <https://getbootstrap.com/docs/5.3/> [1]
9. GNU Sed Manual — <https://www.gnu.org/software/sed/manual/sed.html> [5]
10. GNU Awk Manual — <https://www.gnu.org/software/gawk/manual/gawk.html> [3]

11. GNU Coreutils (sort, cut, head, tail) — <https://www.gnu.org/software/coreutils/manual/coreutils.html> [4]
12. Bash Reference Manual — <https://www.gnu.org/software/bash/manual/bash.html> [2]
13. ANSI Escape Codes — https://en.wikipedia.org/wiki/ANSI_escape_code [17]
14. Snake Game (Wikipedia) — [https://en.wikipedia.org/wiki/Snake_\(video_game_genre\)](https://en.wikipedia.org/wiki/Snake_(video_game_genre)) [18]
15. Libresprite — <https://libresprite.github.io/> [7]
16. Python datetime Module — <https://docs.python.org/3.10/library/datetime.html> [15]
17. Flask Tutorial (Real Python) — <https://realpython.com/tutorials/flask/> [16]
18. Flask Mega-Tutorial by Miguel Grinberg — <https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-i-hello-world> [6]

References

- [1] Bootstrap Team. Bootstrap 5 documentation, 2024.
- [2] Free Software Foundation. Bash reference manual, 2024.
- [3] Free Software Foundation. GNU awk manual, 2024.
- [4] Free Software Foundation. GNU coreutils manual, 2024.
- [5] Free Software Foundation. GNU sed manual, 2024.
- [6] Miguel Grinberg. The flask mega-tutorial part I: Hello, world!, 2024.
- [7] Libresprite contributors. Libresprite, 2024.
- [8] MDN Web Docs. Canvas API, 2024.
- [9] MDN Web Docs. Fetch API, 2024.
- [10] MDN Web Docs. JSON.stringify, 2024.
- [11] MDN Web Docs. KeyboardEvent, 2024.
- [12] MDN Web Docs. setInterval, 2024.
- [13] Pallets Projects. Flask documentation, 2024.
- [14] Pallets Projects. Flask quickstart guide, 2024.
- [15] Python Software Foundation. Python `datetime` module, 2024.
- [16] Real Python. Flask tutorial, 2024.
- [17] Wikipedia contributors. ANSI escape codes, 2024.
- [18] Wikipedia contributors. Snake (video game genre), 2024.